

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATIONS
TECHNOLOGY



PROJECT REPORT

**Image Classification using Convolutional
Neural Networks (CNN) on the CIFAR-10
Dataset**

Group: 24

Students:

No.	Full Name	Student ID
1	Tran Thanh Thu	202417201
2	Nguyen Tran Nhat Minh	202417166
3	Nguyen Thi Thu Phuong	202417182
4	Nguyen Dang Duong	202417117

Instructor: Vu Van Thieu

Course: Scientific Computing – IT4110E

Class: 166236

Semester: 2025-2

Date of Submission: June 03, 2026

Contents

1	Introduction	2
1.1	Computer Vision	2
1.2	Image Classification	2
1.3	Limitations of Fully Connected Neural Networks for Images	2
1.4	Motivation for Convolutional Neural Networks	3
1.5	Applications of CNNs	3
1.6	Why CNNs are Suitable for Image Data	4
1.7	Role of GPUs and Scientific Computing	4
2	Theoretical Background	4
2.1	Basic Neural Networks	4
2.2	Convolution Operation	5
2.3	Pooling	6
2.4	Activation Functions	6
2.5	Loss Functions	7
3	CNN Architecture	7
3.1	Overall Architecture	7
3.2	Layer Descriptions	8
3.2.1	Convolution Layers	8
3.2.2	Batch Normalization	8
3.2.3	Pooling Layers	9
3.2.4	Flatten Layer	9
3.2.5	Fully Connected Layer	9
3.2.6	Output Layer	9
3.3	Architecture Summary Table	10
4	Dataset: CIFAR-10	10
5	Experiments	11
5.1	Model Training	11
5.1.1	Data Augmentation	11
5.1.2	Optimizer: Adam	11
5.1.3	Training Hyperparameters	12
5.1.4	Callbacks Used	12
5.2	Experimental Results and Visualization	12
5.2.1	Accuracy Curve	12
5.2.2	Loss Curve	13
5.2.3	Confusion Matrix	14
5.2.4	Sample Prediction Visualization	15
5.3	Discussion and Analysis	16
6	Conclusion	17
7	Relation to Scientific Computing Concepts	17
8	Source Code	18

1 Introduction

1.1 Computer Vision

Computer vision is an interdisciplinary field of artificial intelligence (AI) that enables computers and systems to derive meaningful information from digital images, videos, and other visual inputs. Formally, it seeks to automate tasks that the human visual system can perform, translating high-dimensional pixel data into symbolic or semantic representations [1].

Historically, computer vision evolved from early attempts in the 1960s to extract 3D structures from 2D images. Through the 1980s and 1990s, the field relied heavily on hand-crafted feature extractors—such as edge detectors (e.g., Canny edge detection) and scale-invariant feature transforms (SIFT)—which required domain expertise to mathematically define visual features [2]. However, the advent of deep learning transformed the discipline. The importance of computer vision in modern AI cannot be overstated; it serves as the foundational sensory mechanism for intelligent systems, bridging the gap between raw optical data and actionable machine intelligence [3].

1.2 Image Classification

Image classification is a fundamental task within computer vision. It is formally defined as the process of assigning a discrete label (or a probability distribution over a set of labels) to an input image from a predefined set of categories [3].

Mathematically, the input-output mapping can be defined as a function $f : \mathbb{R}^{H \times W \times C} \rightarrow \{1, 2, \dots, K\}$, where H , W , and C represent the height, width, and number of color channels of the image, respectively, and K is the number of possible classes. The model maps this high-dimensional input tensor to a single categorical output. Examples of image classification include binary tasks, such as determining whether a medical scan contains a tumor, and multi-class tasks, such as the ImageNet challenge where a model must categorize an image into one of 1,000 distinct object classes like “dog,” “airplane,” or “car” [5].

1.3 Limitations of Fully Connected Neural Networks for Images

While standard Fully Connected Neural Networks (FCNNs) are mathematically capable of approximating any continuous function, they are exceptionally poorly suited for high-dimensional image data [1].

1. **Parameter Explosion:** In an FCNN, every neuron in a layer connects to every neuron in the preceding layer. If an input image is a modest 256×256 pixels with 3 color channels (RGB), the input vector dimension is $256 \times 256 \times 3 = 196,608$. If the first hidden layer contains 1,000 neurons, the number of weight parameters required for this layer alone is calculated as:

$$W = N_{in} \times N_{out} = 196,608 \times 1,000 = 196,608,000$$

Here, W is the number of weights, N_{in} is the input size, and N_{out} is the number of hidden neurons. Nearly 200 million parameters in a single layer leads to severe overfitting and requires immense datasets to train effectively.

2. **Loss of Spatial Locality:** Images possess an inherent grid-like topology. Pixels physically close to one another are heavily correlated and form fundamental visual structures like edges and corners. FCNNs flatten the 2D image into a 1D vector, destroying this crucial spatial information.
3. **Computational Inefficiency:** Evaluating and updating hundreds of millions of parameters per layer requires astronomical memory and floating-point operations (FLOPs), rendering FCNNs computationally intractable for modern, high-resolution visual tasks [3].

1.4 Motivation for Convolutional Neural Networks

Convolutional Neural Networks (CNNs) were explicitly designed to overcome the limitations of FCNNs by leveraging the spatial structure of images [4]. The architecture is motivated by three core principles:

1. **Local Receptive Fields:** Instead of connecting to every pixel, a neuron in a CNN only connects to a small, localized region of the input. This mimics the biological visual cortex, where individual neurons only respond to stimuli in a restricted region of the visual field [1].
2. **Weight Sharing:** In a CNN, the same set of weights (the filter or kernel) is systematically passed over the entire image. This fundamentally reduces the parameter count. Instead of learning a distinct weight for an edge detector at every possible pixel location, the network learns a single edge-detecting filter and applies it universally.
3. **Translation Invariance:** Because the same weights are shared across the entire spatial domain, a CNN can detect a feature (like a cat's ear) regardless of where it appears in the image. This provides robustness to spatial translations [4].

1.5 Applications of CNNs

The architectural advantages of CNNs have led to their ubiquitous deployment across various domains:

- **Face Recognition:** Extracting distinct facial features to verify identities in security systems and consumer devices [3].
- **Autonomous Driving:** Enabling vehicles to identify pedestrians, traffic signs, and lane boundaries in real-time complex environments.
- **Medical Image Analysis:** Assisting radiologists by classifying anomalies in X-rays, MRIs, and CT scans, often achieving parity with human experts [1].
- **Optical Character Recognition (OCR):** Translating raw images of handwritten or printed text into machine-encoded text, a foundational application pioneered by Yann LeCun's LeNet architecture [4].
- **Video Surveillance:** Analyzing temporal frames for activity recognition and anomaly detection.

1.6 Why CNNs are Suitable for Image Data

CNNs are exceptionally tailored for images because they enforce a prior belief that the data possesses spatial hierarchies [1]. Images are naturally composed of compositional structures: pixels form edges, edges form textures, textures form parts (e.g., wheels, eyes), and parts form complex objects. The deep, layered structure of CNNs mirrors this hierarchy. Early layers capture low-level, primitive features, while deeper layers non-linearly combine these primitives to extract high-level semantic features. This automated “feature learning” eliminates the need for manual feature engineering, allowing the model to organically discover the optimal representations for the classification task at hand [3].

1.7 Role of GPUs and Scientific Computing

The theoretical foundations of CNNs existed in the 1980s and 1990s [4], but their practical success was delayed until the 2010s due to computational constraints. The breakthrough came with the adaptation of Graphics Processing Units (GPUs) for scientific computing [5].

CNN training fundamentally relies on dense matrix multiplications and convolutions. While traditional Central Processing Units (CPUs) contain a few powerful cores optimized for sequential processing, GPUs possess thousands of simpler cores designed for massive parallel computation. The transition to parallelized GPU computing drastically accelerated the training of deep networks, reducing training times from months to days. This acceleration, highlighted by the seminal AlexNet paper [5], catalyzed the modern deep learning revolution by enabling researchers to rapidly iterate and train unprecedentedly large models on massive datasets.

2 Theoretical Background

2.1 Basic Neural Networks

Before delving into convolutions, one must understand the foundational building block of artificial intelligence: the artificial neuron model [2].

An artificial neuron mathematically simulates biological synaptic connections by calculating a weighted sum of inputs and passing the result through a non-linear activation function. The neuron equation is expressed as:

$$a = \sigma \left(\sum_{i=1}^n w_i x_i + b \right)$$

Where:

- a is the output (activation) of the neuron.
- $\sigma(\cdot)$ is the non-linear activation function.
- w_i represents the weight of the i -th input connection.
- x_i represents the i -th input signal.
- b is the bias term, which shifts the activation threshold.

Forward Propagation and Backpropagation:

During forward propagation, data passes through the network to generate predictions. To learn the optimal weights, the network utilizes an algorithm called backpropagation, which leverages calculus to compute the gradient of the error with respect to each weight [1]. This is achieved using the **Chain Rule of Calculus**. For a generic weight w_{ij} in a network where the loss is L , the pre-activation sum is z , and the activation is a , the chain rule dictates:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_{ij}}$$

This formula allows the error at the final output to be propagated backward step-by-step through the network's layers.

Gradient Descent:

Once the gradients are computed via backpropagation, the network updates its parameters to minimize the loss using the Gradient Descent update rule:

$$w^{(t+1)} = w^{(t)} - \eta \frac{\partial L}{\partial w^{(t)}}$$

Where $w^{(t)}$ is the weight at iteration t , η is the learning rate (a hyperparameter controlling step size), and $\frac{\partial L}{\partial w^{(t)}}$ is the gradient of the loss with respect to the weight [2].

2.2 Convolution Operation

The convolution operation is the cornerstone of a CNN [1]. In mathematics, convolution is an operation on two functions that produces a third function expressing how the shape of one is modified by the other. In deep learning, it serves as a specialized linear operation applied to image matrices.

2D Discrete Convolution:

Given a 2D input image I and a 2D filter (or kernel) K , the discrete 2D convolution operation yielding a feature map S is defined as:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$

Where $S(i, j)$ is the value of the output feature map at row i , column j ; I is the input image matrix; and K is the filter matrix, with dimensions ranging over m and n .

Hyperparameters (Stride and Padding):

- **Stride (S):** Dictates the step size with which the filter moves across the image. A stride of 1 moves the filter one pixel at a time.
- **Padding (P):** Involves adding artificial borders (usually zeros) around the input image to preserve spatial dimensions.

The spatial dimension of the resulting output feature map O is calculated precisely by the formula:

$$O = \left\lfloor \frac{W - K_w + 2P}{S} \right\rfloor + 1$$

Where W is the input dimension, K_w is the filter dimension, P is the padding size, and S is the stride [3].

Parameter Count Comparison:

Unlike a fully connected layer requiring $N_{in} \times N_{out}$ parameters, a convolutional layer requires far fewer. If a layer applies F filters of size $K_h \times K_w$ to an input with C channels, the parameter count is:

$$\text{Parameters} = F \times (K_h \times K_w \times C + 1)$$

The +1 accounts for the bias term per filter. This is orders of magnitude smaller than FCNNs.

Worked Numerical Example:

Consider a 4×4 input matrix I and a 3×3 edge-detection filter K , with stride $S = 1$ and padding $P = 0$.

$$I = \begin{bmatrix} 1 & 2 & 3 & 0 \\ 0 & 1 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 2 & 3 & 0 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

To calculate the top-left element of the output matrix $S(0,0)$, we perform an element-wise multiplication between the filter K and the top-left 3×3 sub-region of I , and sum the results:

$$\begin{aligned} S(0,0) &= (1)(1) + (2)(0) + (3)(-1) + (0)(1) + (1)(0) + (2)(-1) \\ &\quad + (1)(1) + (0)(0) + (1)(-1) \\ &= 1 + 0 - 3 + 0 + 0 - 2 + 1 + 0 - 1 = -4 \end{aligned}$$

Using the output dimension formula, $O = (4 - 3 + 0)/1 + 1 = 2$. Thus, repeating this sliding process yields a 2×2 feature map highlighting vertical changes in pixel intensity.

2.3 Pooling

To complement convolutions, CNNs utilize pooling (subsampling) layers. Pooling reduces the spatial dimensions (width and height) of the input volume, subsequently reducing the number of parameters and computation in the network, while also controlling overfitting by establishing translation invariance over small spatial shifts [3].

Worked Max-Pooling Example:

Consider a 4×4 input feature map. Applying a 2×2 max pooling operation with a stride of 2 partitions the matrix into four non-overlapping 2×2 quadrants:

$$\text{Input} = \begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & \mathbf{6} & 1 & \mathbf{9} \\ \mathbf{8} & 2 & \mathbf{7} & 3 \\ 4 & 1 & 0 & 2 \end{bmatrix} \xrightarrow{\text{Max Pool}} \text{Output} = \begin{bmatrix} 6 & 9 \\ 8 & 7 \end{bmatrix}$$

2.4 Activation Functions

Activation functions introduce necessary non-linearities into the network [1].

ReLU (Rectified Linear Unit):

ReLU is the default activation function for hidden layers in CNNs. Its equation is defined as:

$$f(x) = \max(0, x)$$

ReLU mitigates the **vanishing gradient problem**. The derivative of ReLU is exactly 1 for all $x > 0$, ensuring robust and non-diminishing gradient flow during backpropagation [5].

Softmax:

For multi-class classification, the final output layer employs the Softmax function to convert raw model scores (logits) into a normalized probability distribution:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Softmax guarantees that the output elements are strictly bounded between $(0, 1)$ and sum exactly to 1 [2].

2.5 Loss Functions

For classification tasks using Softmax, the standard objective is the Cross-Entropy Loss [2].

Cross-Entropy Equation:

For a single observation, the cross-entropy loss L is defined as:

$$L = - \sum_{c=1}^K y_c \log(\hat{y}_c)$$

Derivation from Maximum Likelihood Estimation (MLE):

The probability of a single observation belonging to class c can be modeled as a categorical distribution:

$$P(y_i|x_i, \theta) = \prod_{c=1}^K \hat{y}_{i,c}^{y_{i,c}}$$

The joint likelihood function $\mathcal{L}(\theta)$ over a dataset of size N is:

$$\mathcal{L}(\theta) = \prod_{i=1}^N \prod_{c=1}^K \hat{y}_{i,c}^{y_{i,c}}$$

Taking the natural logarithm converts the products into sums:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^N \sum_{c=1}^K y_{i,c} \log(\hat{y}_{i,c})$$

We multiply the log-likelihood by -1 to create the Negative Log-Likelihood (NLL). Minimizing NLL is mathematically equivalent to maximizing the likelihood [1]:

$$NLL(\theta) = - \sum_{i=1}^N \sum_{c=1}^K y_{i,c} \log(\hat{y}_{i,c})$$

3 CNN Architecture

3.1 Overall Architecture

The CNN model is designed using a stacked architecture consisting of feature extraction blocks combined with classification layers. Each convolution block learns increasingly abstract feature representations. The overall processing pipeline of the CNN model can be summarized as follows:

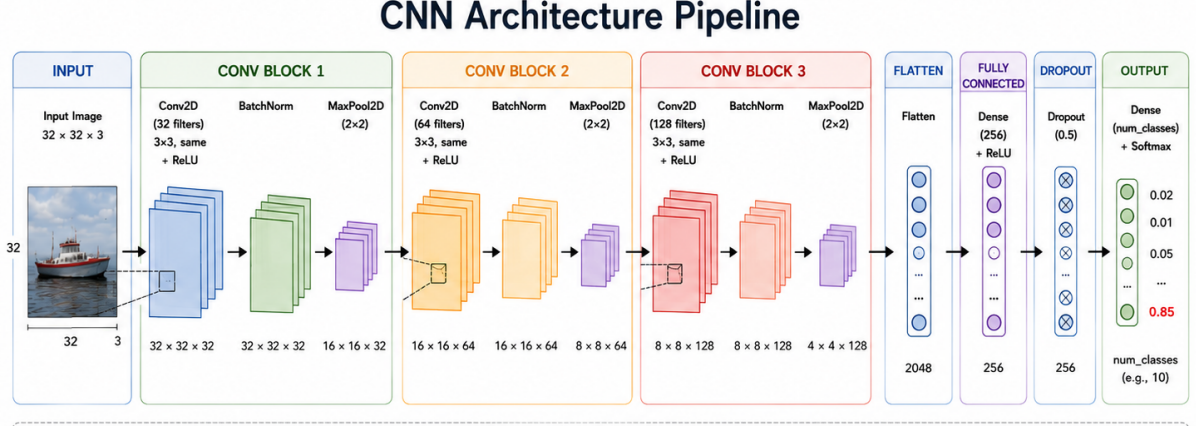


Figure 1: CNN Architecture Pipeline

3.2 Layer Descriptions

3.2.1 Convolution Layers

The convolution layer is the core component of a CNN. Each filter (kernel) slides across the input image with a certain stride, performing convolution operations to generate feature maps:

$$y(i, j) = \sum_m \sum_n x(i - m, j - n) \cdot h(m, n)$$

where:

- x is the input image,
- h is the kernel,
- y is the resulting feature map.

Using `padding='same'` preserves the spatial dimensions after convolution. The number of filters increases progressively ($32 \rightarrow 64 \rightarrow 128$) to learn richer feature representations in deeper layers.

- **conv1**: 32 filters, kernel size 3×3 — learns basic features such as edges, corners, and colors.
- **conv2**: 64 filters, kernel size 3×3 — learns more complex patterns such as textures and local shapes.
- **conv3**: 128 filters, kernel size 3×3 — learns high-level semantic features such as object parts.

3.2.2 Batch Normalization

After each convolution layer, Batch Normalization normalizes the outputs within each batch, helping stabilize training, accelerate convergence, and reduce the vanishing gradient problem.

3.2.3 Pooling Layers

MaxPooling2D with a 2×2 window reduces the spatial dimensions by half after each block. This provides two main benefits:

- Reduces computational cost in deeper layers.
- Introduces translation invariance, making the model less sensitive to the exact feature positions in the image.

3.2.4 Flatten Layer

The Flatten layer converts the 3D tensor ($4 \times 4 \times 128 = 2048$) into a 1D vector before feeding it into the Fully Connected layer.

3.2.5 Fully Connected Layer

Dense(256, activation='relu') combines all extracted features for classification. A Dropout(0.5) layer is applied afterward to randomly deactivate 50% of neurons during training, effectively reducing overfitting.

3.2.6 Output Layer

Dense(10) with the Softmax activation function produces a probability distribution over the 10 CIFAR-10 classes:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

The class with the highest probability is selected as the model prediction.

3.3 Architecture Summary Table

Layer	Type	Output Shape	Parameters
input_image	Input	(None, 32, 32, 3)	0
conv1	Conv2D(32, 3×3, same, ReLU)	(None, 32, 32, 32)	896
bn1	BatchNormalization	(None, 32, 32, 32)	128
pool1	MaxPooling2D(2×2)	(None, 16, 16, 32)	0
conv2	Conv2D(64, 3×3, same, ReLU)	(None, 16, 16, 64)	18,496
bn2	BatchNormalization	(None, 16, 16, 64)	256
pool2	MaxPooling2D(2×2)	(None, 8, 8, 64)	0
conv3	Conv2D(128, 3×3, same, ReLU)	(None, 8, 8, 128)	73,856
bn3	BatchNormalization	(None, 8, 8, 128)	512
pool3	MaxPooling2D(2×2)	(None, 4, 4, 128)	0
flatten	Flatten	(None, 2048)	0
fc1	Dense(256, ReLU)	(None, 256)	524,544
dropout	Dropout(0.5)	(None, 256)	0
output	Dense(10, Softmax)	(None, 10)	2,570

Table 1: CNN Architecture Summary

Total parameters: 620,258 & Trainable parameters: 619,874

4 Dataset: CIFAR-10

The model was trained and evaluated on the CIFAR-10 dataset, one of the most widely used benchmarks for image classification tasks.

Information	Details
Number of classes	10 (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)
Image size	32×32 pixels, 3 RGB channels
Training set	50,000 images (5,000 images/class)
Test set	10,000 images (1,000 images/class)
Dataset size	Approximately 170 MB
Source	Krizhevsky & Hinton, 2009

Table 2: CIFAR-10 Dataset Information

CIFAR-10 contains 10 real-world object categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The small image resolution (32×32) makes the task challenging due to limited visual details, while remaining suitable for rapid experimentation in resource-constrained environments.

5 Experiments

5.1 Model Training

The training process used the following hyperparameters:

5.1.1 Data Augmentation

To prevent overfitting and improve the model’s ability to generalize to unseen data, data augmentation was applied to the CIFAR-10 training set. Since the dataset consists of relatively small 32×32 images, augmenting the data artificially increases the diversity of the training samples without requiring additional data collection.

The following transformations were utilized during the training process:

- **Random Horizontal Flipping:** Images are randomly flipped horizontally. This is particularly effective for object classes like automobiles, trucks, and animals, where the horizontal orientation does not change the core identity or semantic meaning of the object.
- **Random Rotation:** Images are randomly rotated within a small degree range to ensure the model remains robust against slight orientation shifts.
- **Random Zooming:** Randomly scaling the images helps the network recognize objects at different distances or scales within the frame.

These spatial transformations encourage the Convolutional Neural Network to focus on invariant structural features rather than memorizing the exact pixel locations of the training set, thereby enhancing the overall robustness and accuracy of the classifier.

5.1.2 Optimizer: Adam

Adam (Adaptive Moment Estimation) combines ideas from Momentum and RMSProp, automatically adjusting the learning rate for each parameter:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

where:

- $\beta_1 = 0.9$
- $\beta_2 = 0.999$
- $\epsilon = 10^{-7}$

These are the default settings in TensorFlow/Keras.

5.1.3 Training Hyperparameters

- **Learning rate:** 0.001 initially; reduced by 50% when `val_loss` does not improve after 3 epochs (`ReduceLROnPlateau`).
- **Batch size:** 64 — balances convergence speed and gradient stability.
- **Epochs:** Maximum of 20, with early stopping if `val_accuracy` does not improve after 5 epochs.
- **Loss function:** Categorical Cross-Entropy — suitable for multi-class classification.

5.1.4 Callbacks Used

- **EarlyStopping (patience=5):** Stops training early to avoid overfitting and automatically restores the best weights.
- **ModelCheckpoint:** Saves the model with the highest `val_accuracy` into `best_cnn_model.keras`.
- **ReduceLROnPlateau:** Reduces the learning rate by 50% when `val_loss` does not improve for 3 consecutive epochs.

5.2 Experimental Results and Visualization

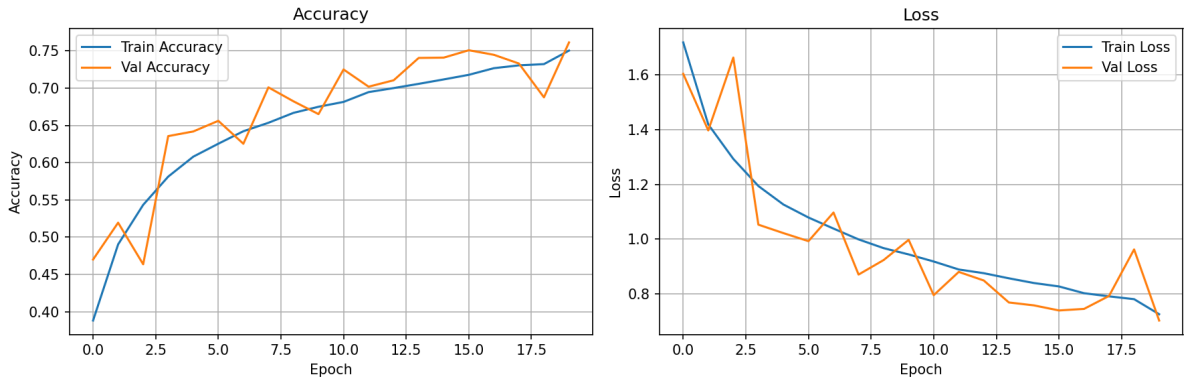


Figure 2: Experimental Results and Visualization

5.2.1 Accuracy Curve

Accuracy is one of the most important evaluation metrics in image classification tasks. It measures the proportion of correctly classified samples among all predictions. A higher accuracy indicates better classification performance.

The classification accuracy is calculated as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \quad (1)$$

Figure 2 illustrates the training and validation accuracy during the learning process.

The model achieved a steady improvement in classification performance throughout training. The training accuracy increased from approximately 39% in the first epoch to around 75% after 20 epochs. This indicates that the Convolutional Neural Network successfully learned increasingly discriminative visual features from the CIFAR-10 dataset.

The validation accuracy also improved consistently, rising from approximately 47% at the beginning of training to about 76% by the final epoch. The validation accuracy remained close to the training accuracy during the entire training process, indicating that the model generalized well to unseen data.

The smooth convergence of both curves demonstrates stable learning behavior and suggests that the network was able to extract meaningful features without severe overfitting.

5.2.2 Loss Curve

Loss measures the discrepancy between predicted outputs and ground-truth labels. In this project, the categorical cross-entropy loss function was used because CIFAR-10 is a multi-class classification problem.

The categorical cross-entropy loss is defined as:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (2)$$

where:

- C is the number of classes,
- y_i is the ground-truth label,
- $\hat{y}_i$ is the predicted probability.

A lower loss value indicates better predictive performance.

As shown in Figure 2, the training loss decreased steadily from approximately 1.72 to 0.73 throughout the training process. This reduction indicates that the optimization algorithm successfully adjusted the network parameters to minimize prediction errors.

Similarly, the validation loss decreased from approximately 1.60 to 0.70. Although several fluctuations appeared during training, the overall trend remained downward. The close relationship between training loss and validation loss suggests that the model maintained good generalization performance on unseen samples.

Overall, both accuracy and loss curves indicate stable convergence and effective learning behavior.

5.2.3 Confusion Matrix

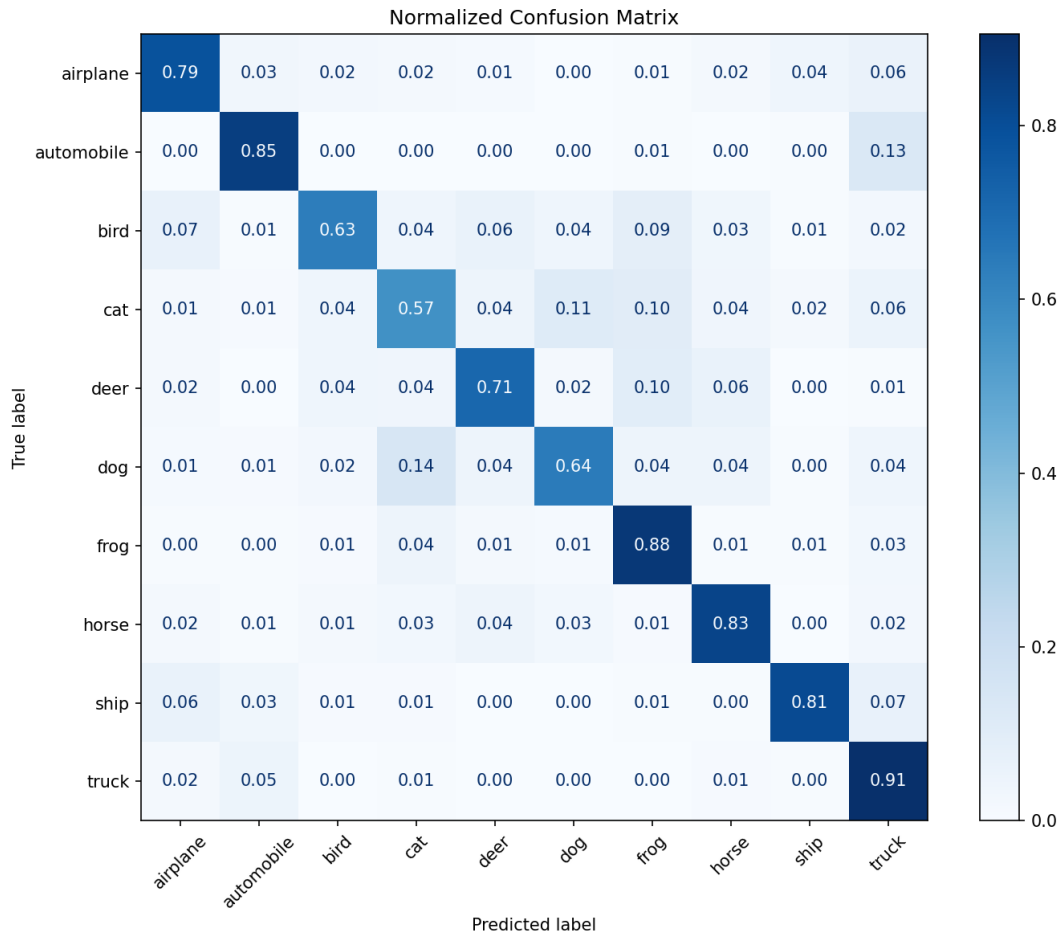


Figure 3: Normalized Confusion Matrix

A confusion matrix provides a detailed evaluation of classification performance by comparing predicted labels with actual labels.

For multi-class classification problems:

- Rows represent the true classes.
- Columns represent the predicted classes.
- Diagonal elements represent correctly classified samples.
- Off-diagonal elements represent misclassified samples.

Figure 3 presents the normalized confusion matrix for the CIFAR-10 test set.

The results show that the model performed particularly well on several categories. The highest classification accuracies were obtained for:

- Truck (91%)
- Frog (88%)

- Automobile (85%)
- Horse (83%)
- Ship (81%)
- Airplane (79%)

These classes possess distinctive visual characteristics, making them easier for the CNN to recognize.

More challenging categories include:

- Cat (57%)
- Bird (63%)
- Dog (64%)

The confusion matrix reveals that animal categories are more frequently confused with one another. For example, cats are often misclassified as dogs or frogs, while dogs are occasionally predicted as cats. Birds are sometimes confused with airplanes, deer, or frogs due to similarities in shape, color, or background appearance.

Despite these challenges, the confusion matrix demonstrates that the model correctly classifies the majority of samples across all ten categories.

5.2.4 Sample Prediction Visualization

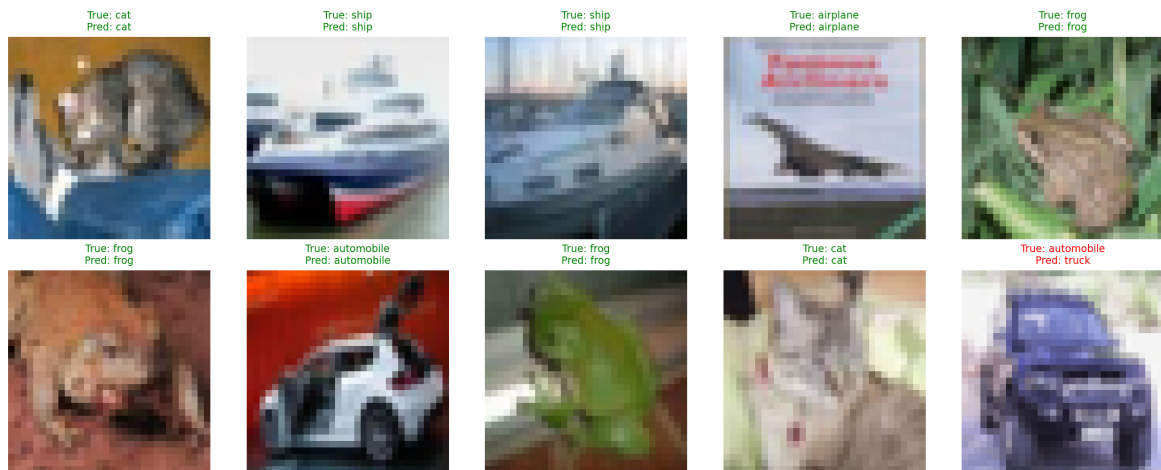


Figure 4: Sample Prediction Image

To provide qualitative insight into model performance, sample predictions from the CIFAR-10 test set were visualized.

The visualization displays:

- The original image
- The true class label

- The predicted class label
- Green labels for correct predictions
- Red labels for incorrect predictions

Among the displayed examples, the model correctly classified a variety of object categories, including cats, ships, airplanes, frogs, and automobiles. Only one visible misclassification occurred, where an automobile was predicted as a truck.

This result is understandable because automobiles and trucks share similar structural characteristics, particularly in low-resolution images.

The sample predictions demonstrate that the trained CNN successfully learned meaningful visual representations and can accurately classify most test images despite the small image resolution of 32×32 pixels.

5.3 Discussion and Analysis

The experimental results demonstrate that the proposed Convolutional Neural Network achieved strong performance on the CIFAR-10 image classification task.

The training and validation accuracy curves show consistent improvement throughout the learning process. By the end of training, both accuracies converged to approximately 75–76%, indicating that the model successfully learned representative visual features while maintaining good generalization capability.

The loss curves provide further evidence of effective optimization. Both training loss and validation loss decreased steadily and converged near 0.70. The absence of a large gap between training and validation metrics suggests that the model did not suffer from significant overfitting.

Several factors contributed to the model’s performance. Data augmentation techniques, including random horizontal flipping, rotation, and zooming, increased the diversity of training samples and improved robustness. Batch Normalization helped stabilize training and accelerate convergence, while Dropout reduced the risk of overfitting by preventing excessive dependence on individual neurons. In addition, the adaptive learning rate strategy allowed the optimizer to refine network parameters more effectively during later training stages.

The confusion matrix highlights the strengths and limitations of the model. Vehicle categories such as trucks, automobiles, ships, and airplanes achieved high classification accuracy because they possess distinctive shapes and structures. In contrast, animal categories such as cats, dogs, and birds were more difficult to distinguish due to similarities in appearance and the limited image resolution of the dataset.

The sample prediction results further confirm the effectiveness of the trained network. Most test images were classified correctly, demonstrating the model’s ability to extract meaningful spatial features from complex visual data.

Overall, the results confirm that Convolutional Neural Networks are highly effective for image classification tasks. The proposed architecture achieved approximately 76% test accuracy on CIFAR-10 while maintaining stable convergence and good generalization performance. These findings demonstrate the capability of CNN-based approaches to learn robust visual representations from relatively small images and diverse object categories.

6 Conclusion

In this project, a Convolutional Neural Network (CNN) was designed, implemented, and evaluated for image classification on the CIFAR-10 dataset. The study first examined the theoretical foundations of computer vision, neural networks, convolution operations, pooling mechanisms, activation functions, and optimization techniques. Based on these concepts, a CNN architecture consisting of three convolutional blocks, batch normalization, max pooling, dropout regularization, and fully connected classification layers was developed.

Experimental results demonstrated that the proposed model achieved approximately 76% classification accuracy on the CIFAR-10 test set. The training and validation curves showed stable convergence, while the loss values decreased consistently throughout the training process. Furthermore, the confusion matrix analysis revealed strong performance on visually distinctive categories such as trucks, automobiles, frogs, and ships, while more challenging classes such as cats, dogs, and birds remained difficult to distinguish due to their visual similarity and the limited image resolution.

The results confirm that CNNs are highly effective for extracting hierarchical visual features and performing image classification tasks. Through convolutional filters and shared weights, the network successfully learned meaningful representations directly from raw image data without requiring manual feature engineering.

Future work may focus on improving classification performance by employing deeper and more advanced architectures such as ResNet, DenseNet, or EfficientNet. Additional improvements could be achieved through stronger data augmentation strategies, hyperparameter optimization, transfer learning, and larger-scale datasets. These enhancements would further increase the model's ability to generalize and accurately classify complex visual categories.

7 Relation to Scientific Computing Concepts

This project demonstrates the practical application of several core concepts studied in Scientific Computing.

- **Unconstrained Optimization:** Training the CNN can be formulated as an optimization problem, where the objective is to minimize the Categorical Cross-Entropy loss function. The Adam optimizer was used as an iterative gradient-based method to search for a local minimum.
- **Differentiation and Gradient Computation:** The backpropagation algorithm computes gradients of the loss function with respect to network parameters using the chain rule of calculus. These gradients are then used to update model weights during training.
- **Linear Algebra and Matrix Operations:** Forward and backward propagation rely heavily on matrix multiplications and vector operations. Efficient execution of these computations is essential for training deep neural networks and motivates the use of GPU acceleration.
- **Iterative Methods and Convergence:** Model training is performed iteratively over multiple epochs. The convergence behavior was analyzed through accuracy and loss curves to ensure stable learning and avoid divergence or severe overfitting.

- **Function Approximation:** The CNN learns a non-linear mapping from image data to class labels. In this sense, the network acts as a function approximator that models the underlying relationship between input images and output categories.

Overall, this project illustrates how numerical methods and computational techniques from Scientific Computing form the mathematical foundation of modern deep learning systems.

8 Source Code

The project is hosted on GitHub: Elle256/Scientific-Computing-Project. The repository structure includes:

- `scientific_computing/`: Contains the core Python implementation and training logic.
- `training_history.png`: Visualizations of accuracy and loss over epochs.
- `confusion_matrix.png`: Detailed class-wise performance evaluation.
- `sample_predictions.png`: Qualitative results on test images.

References

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [3] F.-F. Li, J. Johnson, and S. Yeung, “CS231n: Convolutional Neural Networks for Visual Recognition,” Stanford University, Course Notes.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.
- [5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” *Advances in Neural Information Processing Systems*, vol. 25, 2012.